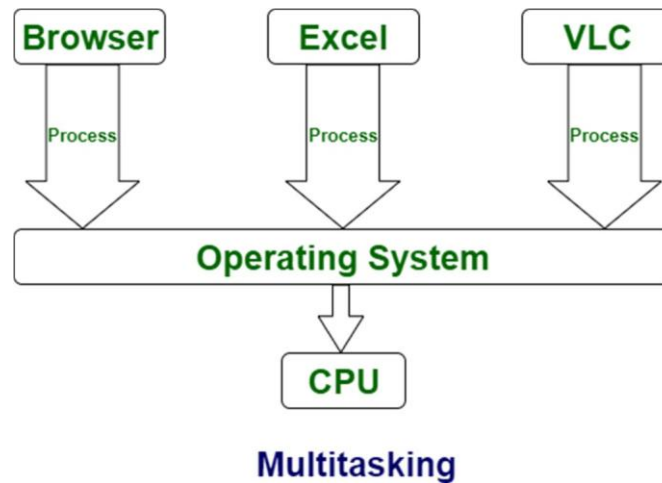


Topic-1: Multithreading in Java-Introduction

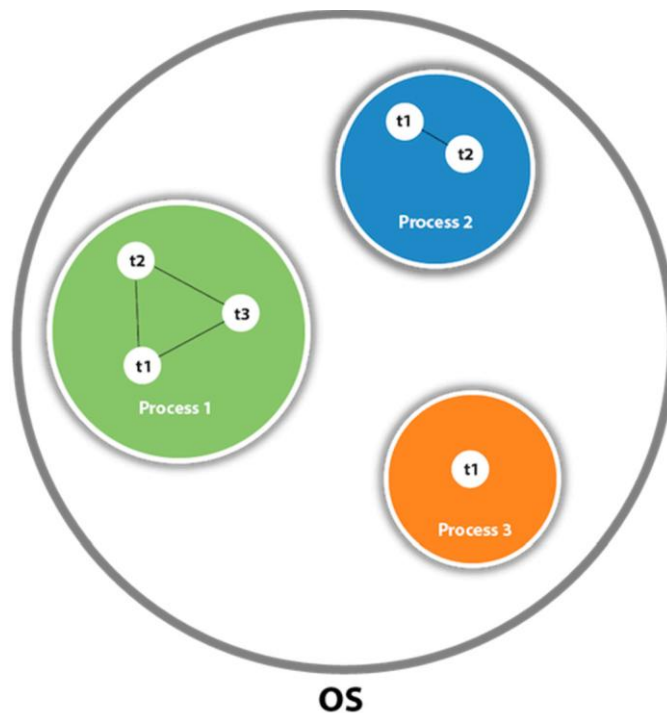
1.1 Multi-Tasking in Java

Multi-tasking is the ability of the CPU to perform multiple tasks simultaneously. Multi-tasking involves often CPU switching between the tasks, so that users can collaborate with each program together. The CPU collaborates between the users and performs multiple tasks effectively. In multi-tasking, each process has a different resource and memory and does not share it.



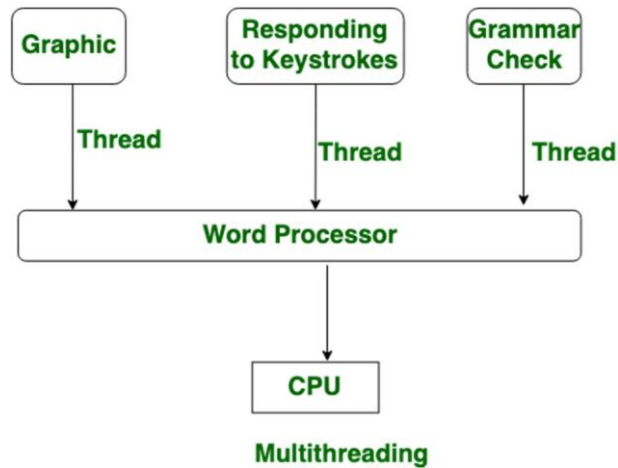
1.2 What is Thread in java?

- A thread is a lightweight sub-process, the smallest unit of processing. It is a separate path of execution.
- Threads are independent i.e., if there occurs exception in one thread, it doesn't affect other threads.
- All threads of a process share the common memory.



1.3 Multi-Threading in Java

- The process of executing multiple threads simultaneously is known as multithreading.
- The main purpose of multi-threading is to provide simultaneous execution of two or more parts of a program to maximum utilize the CPU time.
- A multi-threaded program contains two or more parts that can run concurrently. Each such part of a program called thread.



Differences between Multi-Tasking and Multi-Threading

S.NO	Multitasking	Multithreading
1.	In multitasking, users are allowed to perform many tasks by CPU.	While in multithreading, many threads are created from a process through which computer power is increased.
2.	Multitasking involves often CPU switching between the tasks.	While in multithreading also, CPU switching is often involved between the threads.
3.	In multitasking, the processes share separate memory.	While in multithreading, processes are allocated same memory.
4.	Multitasking component involves multiprocessing.	While multithreading component does not involve multiprocessing.
5.	In multitasking, CPU is provided in order to execute many tasks at a time.	While in multithreading also, CPU is provided in order to execute many threads from a process at a time.
6.	In multitasking, processes don't share same resources, each process is allocated separate resources.	While in multithreading, each process share same resources.
7.	Multitasking is slow compared to multithreading.	While multithreading is faster.
8.	In multitasking, termination of process takes more time.	While in multithreading, termination of thread takes less time.

Topic-2: Thread Class in Java

The `Thread` class in Java is a fundamental class for creating and managing threads. A thread is a separate flow of execution that runs concurrently with the main program.

Constructors

The `Thread` class has several constructors:

1. `Thread()`

Default constructor.

2. `Thread(String name)`

Constructor with thread name.

3. `Thread(Runnable target)`

Constructor with a `Runnable` object.

4. `Thread(Runnable target, String name)`

Constructor with a `Runnable` object and thread name.

Methods

Some key methods of the `Thread` class:

1. `start()`

Starts the thread.

2. `run()`

Method that contains the code to be executed by the thread.

3. `join()`

Waits for the thread to die.

4. `join(long millis)`

Waits for the thread to die for the specified time.

5. `interrupt()`

Interrupts the thread.

6. `isAlive()`

Checks if the thread is alive.

7. `getName()`

Returns the thread name.

8. `setName(String name)`

Sets the thread name.

9. `getId()`

Returns the thread ID.

10. `isInterrupted()`

Checks if the thread is interrupted

Topic-3: Main Thread in Java

In Java, the main thread is the thread that starts the execution of a Java program. It is the first thread created by the Java Virtual Machine (JVM) when a program is launched.

Characteristics

Here are some key characteristics of the main thread:

1. **Default Thread:** The main thread is the default thread created by the JVM.
2. **First Thread:** It is the first thread to start execution in a Java program.
3. **Main Method:** The main thread executes the `public static void main(String[] args)` method.
4. **Thread Name:** The main thread's name is "main".
5. **Priority:** The main thread's priority is 5 (normal priority).
6. **Daemon:** The main thread is a non-daemon thread.

Lifecycle

Here's the lifecycle of the main thread:

1. **Creation:** The JVM creates the main thread when a Java program is launched.
2. **Execution:** The main thread starts executing the `main()` method.
3. **Completion:** When the `main()` method completes, the main thread terminates.
4. **Program Termination:** If the main thread terminates, the JVM terminates the program.

Key Methods

Some key methods related to the main thread:

1. `public static void main(String[] args)`: The entry point of the Java program.
2. `Thread.currentThread()`: Returns the currently executing thread (main thread).
3. `Thread.sleep()`: Puts the main thread to sleep for a specified time.

Example:

```
public class MainThreadExample {
    public static void main(String[] args) {
        System.out.println("Main thread name: " +
Thread.currentThread().getName());
        System.out.println("Main thread priority: " +
Thread.currentThread().getPriority());

        // Main thread execution
        for (int i = 0; i < 5; i++) {
            System.out.println("Main thread is running");
        }
    }
}
```

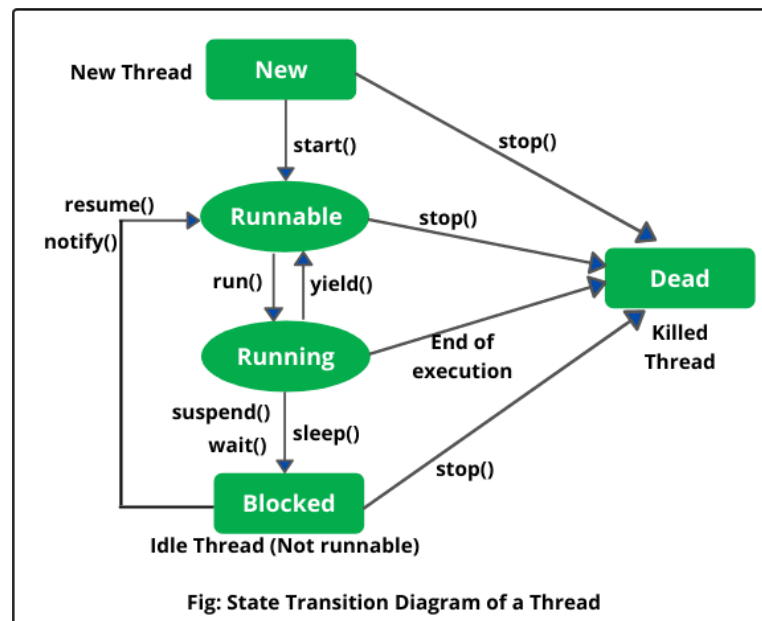
Output:

```
Main thread name: main
Main thread priority: 5
Main thread is running
Main thread is running
Main thread is running
Main thread is terminating
```

Topic-4 THREAD LIFE CYCLE

A thread goes through various stages in its life cycle. A thread in Java at any point of time exists in any one of the following states:

1. New
2. Runnable
3. Running
4. Blocked (Non-runnable state)
5. Dead



1. New (Newborn State): When we create a thread object using Thread class, thread is born and is known to be in Newborn state. That is, when a thread is born, it enters into new state but the start() method has not been called yet on the instance.

2. Runnable state: Runnable state means a thread is ready for execution. When the start() method is called on a new thread, thread enters into a runnable state. In runnable state, thread is ready for execution and is waiting for availability of the processor (CPU time). That is, thread has joined queue (line) of threads that are waiting for execution.

3. Running state: Running means Processor (CPU) has allocated time slot to thread for its execution. When thread scheduler selects a thread from the runnable state for execution, it goes into running state. In running state, processor gives its time to the thread for execution and executes its run method. This is the state where thread performs its actual functions. A thread can come into running state only from runnable state.

4. Blocked state: A thread is considered to be in the blocked state when it is suspended, sleeping, or waiting for some time in order to satisfy some condition.

5. Dead state: A thread dies or moves into dead state automatically when its run() method completes the execution of statements. That is, a thread is terminated or dead when a thread comes out of run() method. A thread can also be dead when the stop() method is called.

Topic-5: CREATING THREADS

There are two ways to create a thread:

1. By extending Thread class
2. By implementing Runnable interface.

1. Creating Thread by extending Thread class

Step-1: Create a thread by defining a new class that extends **Thread** class

Step-2: The extending class must override **run()** method which is the entry point of new thread.

Step-3: Create an instance of that class and invoke **start()** method to call run() method.

Example:

```
class MyThread extends Thread{
    public void run(){
        System.out.println("thread is running...");
    }
    public static void main(String args[]){
        MyThread t1=new MyThread();
        t1.start();
    }
}
```

Output:

```
thread is running...
```

2. Creating Thread by implementing Runnable interface

Step-1: Create a thread by defining a new class that implements **Runnable** interface.

Step-2: The extending class must implement **run()** method which is the entry point of new thread.

Step-3: Create an instance of that class and attach it to Thread class object.

Step-4: Invoke **start()** method on Thread class object to call run() method.

Example:

```
class MyThread implements Runnable{
    public void run(){
        System.out.println("thread is running...");
    }
    public static void main(String args[]){
        MyThread m=new MyThread();
        Thread t1=new Thread(m);
        t1.start();
    }
}
```

Output:

thread is running...

Topic-6 THREAD SYNCHRONIZATION

- Synchronization in java is the capability to control the access of multiple threads to any shared resource.
- Java Synchronization is better option where we want to allow only one thread to access the shared resource.
- Synchronization is mainly used to prevent thread interference and consistency problem.

In java, Thread synchronization is achieved in 2 ways

1. by synchronized block
2. by synchronized method

1. Synchronized block in java

- Synchronized block can be used to perform synchronization on any specific resource of the method.
- Synchronized block is used to lock an object for any shared resource.
- Scope of synchronized block is smaller than the method.

Syntax:

```
synchronized (objectreference) {
    //code block
}
```

Example:

```
class Table{
    void printTable(int n){
        synchronized(this){//synchronized block
            for(int i=1;i<=5;i++){
                System.out.println(n*i);
                try{
                    Thread.sleep(200);
                }
            }
            catch(Exception e){
                System.out.println(e);
            }
        }
    }
}
```

```
        }
    } //for
    } //synchronized
    } //printTable
}

class MyThread1 extends Thread{
    Table t;
    - MyThread1(Table t){
        this.t=t;
    }
    public void run(){
        System.out.println("Thread1 running and printing
table of 5");
        t.printTable(5);
    }
}

class MyThread2 extends Thread{
    Table t;
    MyThread2(Table t){
        this.t=t;
    }
    public void run(){
        System.out.println("Thread2 running and printing
table of 6");
        t.printTable(6);
    }
}

class TestSynchronizedBlock{
    public static void main(String args[]){
        Table obj = new Table();//only one object
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
        t1.start();
        t2.start();
    }
}
```

Output:

```
Thread1 running and printing table of 5
Thread2 running and printing table of 6
5
10
15
20
25
6
12
18
24
```

30

Note: In the above program, if thread synchronization is not implemented, then we get the output as:

```
Thread1 running and printing table of 5
Thread2 running and printing table of 6
5
6
10
12
15
18
24
20
25
30
```

2. Synchronized method in java

- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

Syntax:

```
synchronized returntype methodname (parameter list) {
    //method body
}
```

Example:

```
class Table{
    //making the method as synchronized
    synchronized void printTable(int n){
        for(int i=1;i<=5;i++){
            System.out.println(n*i);
            try{
                Thread.sleep(200);
            }
            catch(Exception e){
                System.out.println(e);
            }
        }
    }
}
```

```
class MyThread1 extends Thread{
    Table t;
    MyThread1(Table t){
        this.t=t;
    }
    public void run(){
```

```
        System.out.println("Thread1 running and printing
table of 5");
        t.printTable(5);
    }
}

class MyThread2 extends Thread{
    Table t;
    MyThread2(Table t){
        this.t=t;
    }
    public void run(){
        System.out.println("Thread2 running and printing
table of 6");
        t.printTable(6);
    }
}

class TestSynchronizedBlock{
    public static void main(String args[]){
        Table obj = new Table();//only one object
        MyThread1 t1=new MyThread1(obj);
        MyThread2 t2=new MyThread2(obj);
        t1.start();
        t2.start();
    }
}
```

Output:

```
Thread1 running and printing table of 5
Thread2 running and printing table of 6
5
10
15
20
25
6
12
18
24
30
```

Topic-7: Inter Thread Communication

Inter-thread communication in Java

- **Inter-thread communication** or **Co-operation** is all about allowing synchronized threads to communicate with each other.
- Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods of **Object class**:

1. wait()
2. notify()
3. notifyAll()

1. wait() method

- Causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

2. notify() method

- Wakes up a single thread that is waiting on this object's monitor.
- If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.

3. notifyAll() method

- Wakes up all threads that are waiting on this object's monitor.

Example :

```
class Customer{
    int amount=10000;

    synchronized void withdraw(int amount){
        System.out.println("going to withdraw"+amount);

        if(this.amount<amount){
            System.out.println("Less balance; waiting for
deposit...");
            try{
                wait();
            }
            catch(Exception e){
                System.out.print(e);
            }
        }//if
        this.amount-=amount;
        System.out.println("withdraw completed...");
    }//withdraw

    synchronized void deposit(int amount){
```

```
        System.out.println("going to deposit "+amount);
        this.amount+=amount;
        System.out.println("deposit completed... ");
        notify();
    } //deposit
} //Customer

class Test{
    public static void main(String args[]){
        Customer c=new Customer();
        new Thread(){
            public void run(){
                c.withdraw(15000);
            }
        }.start();
        new Thread(){
            public void run(){
                c.deposit(10000);
            }
        }.start();
    } //main
} //Test
```

Output:

```
going to withdraw 15000
Less balance; waiting for deposit...
going to deposit 10000
deposit completed...
withdraw completed...
```

Topic-8: Thread Priorities in Java

In Java, thread priority is a way to indicate the relative importance of a thread in relation to other threads. The Java Virtual Machine (JVM) uses thread priority to decide the order in which threads are scheduled for execution. However, it's important to note that thread priority is only a hint to the scheduler; it does not guarantee any specific behavior.

Thread Priority Levels

Java defines several constants in the `Thread` class to represent thread priorities:

- `Thread.MIN_PRIORITY` (value 1): Lowest priority.
- `Thread.NORM_PRIORITY` (value 5): Default priority.
- `Thread.MAX_PRIORITY` (value 10): Highest priority.

Setting Thread Priority

You can set the priority of a thread using the `setPriority(int priority)` method and get the priority using the `getPriority()` method.

Example of Using Thread Priority:

```
class MyThread extends Thread {
    public MyThread(String name) {
        super(name);
    }

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println(Thread.currentThread().getName() + "
with priority " + getPriority() + " - Count: " + i);
            try {
                Thread.sleep(100); // Sleep for demonstration
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}

public class ThreadPriorityExample {
    public static void main(String[] args) {
        MyThread thread1 = new MyThread("Thread 1");
        MyThread thread2 = new MyThread("Thread 2");
        MyThread thread3 = new MyThread("Thread 3");

        // Set priorities
        thread1.setPriority(Thread.MIN_PRIORITY); // Lowest priority
        thread2.setPriority(Thread.NORM_PRIORITY); // Default
priority
        thread3.setPriority(Thread.MAX_PRIORITY); // Highest
priority

        // Start threads
        thread1.start();
        thread2.start();
        thread3.start();
    }
}
```

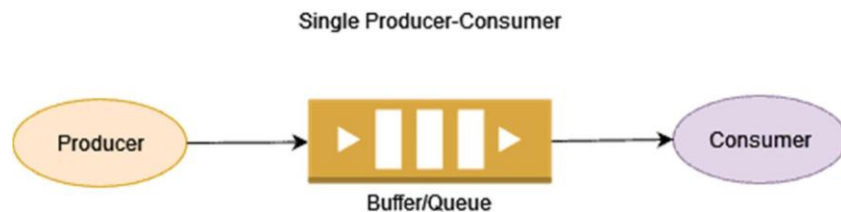
Output:

```
Thread 3 with priority 10 - Count: 0
Thread 2 with priority 5 - Count: 0
Thread 1 with priority 1 - Count: 0
Thread 3 with priority 10 - Count: 1
Thread 2 with priority 5 - Count: 1
Thread 1 with priority 1 - Count: 1
Thread 2 with priority 5 - Count: 2
```

```
Thread 3 with priority 10 - Count: 2
Thread 1 with priority 1 - Count: 2
Thread 3 with priority 10 - Count: 3
Thread 2 with priority 5 - Count: 3
Thread 1 with priority 1 - Count: 3
Thread 3 with priority 10 - Count: 4
Thread 2 with priority 5 - Count: 4
Thread 1 with priority 1 - Count: 4
```

Topic: Producer-Consumer Problem

Producer and Consumer are two separate processes. Both processes share a common buffer or queue. The producer continuously produces certain data and pushes it onto the buffer, whereas the consumer consumes those data from the buffer.



- Both producer and consumer may try to update the queue at the same time. This could lead to data loss or inconsistencies.
- Producers might be slower than consumers. In such cases, the consumer would process elements fast and wait.
- In some cases, the consumer can be slower than the producer. This situation leads to a queue overflow issue.

We need to handle resource sharing and synchronization to solve a few complexities:

- Synchronization on queue while adding and removing data
- When the queue is empty, the consumer has to wait until the producer adds new data to the queue
- When the queue is full, the producer has to wait until the consumer consumes data and the queue has some empty buffer

Program:

```
import java.util.LinkedList;
import java.util.Queue;
import java.util.Random;

public class ProducerConsumerDemo {

    public static void main(String args[]) {
        System.out.println("Solving Producer Consumer Problem");
        Queue<Integer> buffer = new LinkedList<>();
        int maxSize = 3;
        Thread producer = new Producer(buffer, maxSize, "PRODUCER");
        Thread consumer = new Consumer(buffer, maxSize, "CONSUMER");
        producer.start();
        consumer.start();
    }
}
```

```
    }
}

/*
 * Producer Thread will keep producing values for Consumer
 * to consumer. It will use wait() method when Queue is full
 * and use notify() method to send notification to Consumer
 * Thread.
 */
class Producer extends Thread {
    private Queue<Integer> queue;
    private int maxSize;
    public Producer(Queue<Integer> queue, int maxSize, String name){
        super(name);
        this.queue = queue;
        this.maxSize = maxSize;
    }
    @Override
    public void run() {
        while (true) {
            synchronized (queue) {
                while (queue.size() == maxSize) {
                    try {
                        System.out.println("Queue is full, "
                            + "Producer thread waiting for "
                            + "consumer to take something from
queue");
                        queue.wait();
                    } catch (Exception ex) {
                        ex.printStackTrace();
                    }
                }

                Random random = new Random();
                int i = random.nextInt((10 - 1) + 1);
                System.out.println("Producing value : " + i);
                queue.add(i);
                queue.notifyAll();
            }
        }
    }
}

/**
 * Consumer Thread will consumer values form shared queue.
 * It will also use wait() method to wait if queue is
 * empty. It will also use notify method to send
 * notification to producer thread after consuming values
 * from queue.
 */
class Consumer extends Thread {
    private Queue<Integer> queue;
    private int maxSize;

    public Consumer(Queue<Integer> queue, int maxSize, String name){
```

```
        super(name);
        this.queue = queue;
        this.maxSize = maxSize;
    }

    @Override
    public void run() {
        while (true) {
            synchronized (queue) {
                while (queue.isEmpty()) {
                    System.out.println("Queue is empty,"
                        + "Consumer thread is waiting"
                        + " for producer thread to put something
in queue");
                    try {
                        queue.wait();
                    } catch (Exception ex) {
                        ex.printStackTrace();
                    }
                }
                System.out.println("Consuming value : " +
queue.remove());
                queue.notifyAll();
            }
        }
    }
}
```

Output:

```
Solving Producer Consumer Problem
Producing value : 0
Producing value : 9
Producing value : 7
Queue is full, Producer thread waiting for consumer to take
something from queue
Consuming value : 0
Consuming value : 9
Consuming value : 7
Queue is empty, Consumer thread is waiting for producer thread to put
something in queue
Producing value : 0
Producing value : 4
Producing value : 4
Queue is full, Producer thread waiting for consumer to take
something from queue
Consuming value : 0
Consuming value : 4
Consuming value : 4
Queue is empty, Consumer thread is waiting for producer thread to put
something in queue
```